

Knowing Where, Not Going There: Vision-Language-Action Policies Encode Displacements They Do Not Act On

Anonymous CVPR submission

Paper ID *****

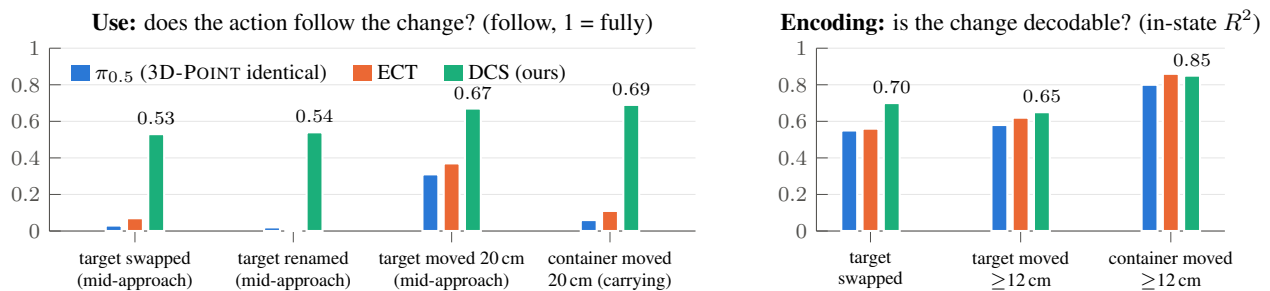


Figure 1. $\pi_{0.5}$ **knows where, but does not go there**. Probes at decision states of LIBERO-Object rollouts (Sec. 3). *Right*: the displacement of the target or the container is decodable from the action tokens of every model. *Left*: at the moments that decide the task, the action of $\pi_{0.5}$ barely follows it; neither equivariant counterfactual training (ECT) nor an injected oracle 3D goal point (3D-POINT, identical to $\pi_{0.5}$) changes that. Decision-time counterfactual supervision (DCS) does, without changing the encoding.

Abstract

001 Vision-language-action (VLA) policies that are near-perfect
 002 on LIBERO collapse once the objects in the scene trade
 003 places: $\pi_{0.5}$ completes 99% of LIBERO-Object episodes but
 004 only 19% after two objects are swapped, driving to where
 005 the target used to be. We ask whether this is a failure of per-
 006 ception or of use. Intervening on the simulator at arbitrary
 007 states of the policy’s own rollouts, we measure, phase by
 008 phase, how far the predicted action follows a displacement
 009 of the target or the container (use) and how well that dis-
 010 placement can be decoded from the policy’s hidden states
 011 (encoding). $\pi_{0.5}$ encodes the displacement (in-state probe
 012 R^2 0.55–0.86), yet at the moments the task is decided its
 013 action follows it by only 2–25%. Two recent remedies leave
 014 use unchanged in a matched setting: equivariant counter-
 015 factual training on transformed demonstrations, and inject-
 016 ing a privileged 3D goal point into the action expert, which
 017 is learned as a constant bias. Supervising the policy in
 018 counterfactual worlds built at its own decision states, with
 019 privileged labels gated for validity, raises use to 0.44–0.69
 020 while encoding stays where it was. Restricting which pa-
 021 rameters may change localizes the effect: adapting only

the VLM backbone, mostly its layers 9–17, recovers about
 90% of the gain, whereas the action expert, its readout, or
 the VLM’s keys and values alone do not. Under the same
 teacher and training budget, this improves LIBERO-PRO
 object swaps by 31.3 points on LIBERO-Object and 14.8
 points on LIBERO-Spatial over equivariant counterfactual
 training, at unchanged nominal success. We also report
 what it costs: a trade-off with appearance changes, and a
 sensitivity to the furniture placement that LIBERO re-draws
 at every reset, an evaluation detail we show moves per-task
 success by up to 25 points.

1. Introduction

Generalist vision-language-action (VLA) policies [8, 9, 17, 24] now solve the standard LIBERO suites [21] almost perfectly. The same policies fail in a strikingly simple way when the scene is rearranged. The *swap* perturbation of LIBERO-PRO [37] keeps every task and only lets the target trade places with another object; under it, $\pi_{0.5}$ drops from 99% to 19% on LIBERO-Object and from 97.5% to 40% on LIBERO-Spatial. Watching the rollouts shows why: the arm drives to where the target used to be. The policy be-

043 has as if the instruction retrieved a memorized trajec- 095
044 tory [10]. 096

045 A natural reading is that the policy does not *perceive* 097
046 where things are, and a large body of work adds percep- 098
047 tion: depth, point clouds, 3D features, or an explicit 3D goal 099
048 point [26, 30, 36]. We test this reading directly and find it 100
049 wrong for $\pi_{0.5}$. Because our policies run in a simulator, we 101
050 can stop any rollout at any state, change exactly one thing 102
051 (move the target, swap it with another object, move the con- 103
052 tainer while the object is carried, rename the target in the 104
053 instruction), re-render, and ask the policy again. Compar- 105
054 ing the two predictions measures how much the action *uses* 106
055 the change; probing the hidden states before and after the 107
056 change measures how well the change is *encoded*. The two 108
057 disagree (Fig. 1). The displacement is linearly decodable 109
058 from the action tokens of $\pi_{0.5}$ (R^2 0.55–0.86), but at the 110
059 moments that decide the task, when the hand is still on its 111
060 way to the target or carries the object to the container, the 112
061 predicted action follows the displacement by only 2–25%. 113
062 $\pi_{0.5}$ *knows where* the target is and does not *go there*. 114

063 This distinction predicts which remedies can work, and 115
064 Sec. 4 makes the prediction precise: on fixed layouts the 116
065 data cannot tell a policy that uses the target’s position from 117
066 one that replays the memorized one. If the information is al- 118
067 ready present, adding more of it should not help, and it does 119
068 not: re-implementing the injection of a privileged 3D goal 120
069 point into the action expert [30] in our setting leaves swap 121
070 success at the base level, and the injected channel is learned 122
071 as a constant offset that ignores the point. Equivariant coun- 123
072 terfactual training (ECT) [10], which pairs demonstrations 124
073 with geometrically transformed replays, diagnoses the same 125
074 memorization but, in a matched setting, leaves the measured 126
075 use where it was. What does change use is supervision *at* 127
076 *the decision states themselves*: building, at a state of the pol- 128
077 icy’s own rollout, a world in which the memorized action is 129
078 wrong (the target or the container has moved) and super- 130
079 vising the correct action there. We call this decision-time 131
080 counterfactual supervision (DCS). It raises use from 0.03– 132
081 0.09 to 0.44–0.69 while the encoding does not change: it 133
082 teaches the policy to act on what it already represents rather 134
083 than to see better. 135

084 The same instrument tells us *where* the change happens. 136
085 Restricting the trainable parameters to different parts of 137
086 $\pi_{0.5}$, with the same counterfactual data, shows that the ac- 138
087 tion expert, its readout layer, or the keys and values the ac- 139
088 tion expert reads from the VLM are not where the fix lives; 140
089 adapting the VLM backbone alone, with the action expert 141
090 frozen, recovers about 90% of the swap gain, and most of 142
091 that comes from its layers 9–17. 143

092 Contributions. 144

- 093 • A decision-time intervention protocol that separates *use* 145
094 from *encoding* in a VLA, per task phase, and the finding

that $\pi_{0.5}$ encodes object and container displacements it 095
does not act on (Sec. 3). 096

- An identifiability account (Sec. 4): when layouts are 097
fixed, the training risk is indifferent to whether a policy 098
uses the positions it encodes, so adding information (*e.g.* 099
a 3D goal point) cannot help, while counterfactuals that 100
move a single decision variable at decision states, with 101
labels that depend on it, identify use. Five predictions of 102
this account hold in our experiments, including that priv- 103
ileged 3D goal injection is ignored (Sec. 6.2). 104
- A localization of the change that does fix use: the VLM’s 105
mid-to-late layers, not the action pathway (Sec. 6.3). 106
- Decision-time counterfactual supervision with validity- 107
gated privileged labels, which improves LIBERO-PRO 108
swap by 31.3 points (LIBERO-Object) and 14.8 points 109
(LIBERO-Spatial) over ECT under the same teacher and 110
budget (Sec. 6.1), together with an account of its costs 111
and of an evaluation detail of LIBERO, the per-reset re- 112
placement of furniture, that changes per-task results by up 113
to 25 points (Sec. 6.4). 114

2. Related Work 115

VLA policies and their generalization. VLAs fine-tune 116
vision-language backbones to output robot actions, as 117
discrete tokens [9, 17] or through a flow-matching ac- 118
tion expert attached to the backbone [8, 24]; continuous- 119
action fine-tuning brought them close to saturation on 120
LIBERO [18, 21]. Robustness benchmarks built on 121
LIBERO show how narrow this success is: LIBERO- 122
Plus [14] perturbs camera, lighting, robot initial state and 123
layout, and LIBERO-PRO [37] swaps and moves objects, 124
rephrases instructions and changes tasks, with near-zero 125
success for most models under position changes. We study 126
the position failures of $\pi_{0.5}$, the strongest open model on 127
these suites. 128

Memorization and shortcuts in imitation. Behavior 129
cloning is known to latch onto spurious correlates of the ex- 130
pert’s action, such as past actions or proprioception [12, 31], 131
an instance of shortcut learning [15]. Gradient-based train- 132
ing can leave predictive features unused [23, 28], and in 133
image classification the core features are often learned yet 134
ignored, so that re-training the last layer on data without the 135
spurious correlation restores robustness [19]. We find the 136
same “learned but unused” state in a VLA, but not the same 137
cure: re-training the action readout does not restore use 138
(Sec. 6.3). For VLAs, recent analyses report that instruc- 139
tions retrieve memorized trajectories [10], that new target 140
positions remain decodable from internal features while the 141
arm goes to the old one [4], and that spatial motor programs 142
are tied to the scene layout [5]. VLA-Trace [29] traces 143
representations and attention pathways of $\pi_{0.5}$ and Open- 144
VLA during fine-tuning. A related line studies *instruction* 145

blindness, where visual priors override language: LIBERO-CF and counterfactual action guidance [13], deconfounding with a language-masked branch [35], residual semantic steering [34], and flatness-preserving fine-tuning [6]. Our measurements agree with the memorization diagnosis; what we add is a per-phase, per-intervention separation of use and encoding that compares models before and after a remedy, and a localization of the parameters whose change restores use.

Adding geometry. Many works feed depth, point clouds or 3D features to VLAs [26, 36]; where position perturbations were measured, they did not resolve them. Closest to our question, Tsai *et al.* [30] inject a grounded 3D goal point into the adaptive normalization of the action expert and report large gains when training on 69 tasks with varied layouts. We re-implement the injection with an oracle point in our per-suite setting and find that the channel is ignored; with counterfactual supervision added, the policy still solves the task through the VLM rather than through the point.

Escaping memorized trajectories. Inference-time methods leave the policy unchanged: Affordance Field Intervention [32] detects “memory traps” and steers $\pi_{0.5}$ with 3D affordance fields and waypoints (reporting +20.2% on LIBERO-PRO), and CoRe [2] plans recoveries in imagination. CounterAlign [3] learns a reward from counterfactual discriminators for offline RL and reports gains under position perturbations. These are complementary to ours: they act at test time or through a learned reward, whereas we ask where in the policy the failure lives and supervise that decision directly.

Counterfactual data and privileged teachers. Counterfactual data augmentation [25] and demonstration generation by replaying transformed trajectories [22, 33] create new supervision from existing episodes. ECT [10] is the closest method: it mirrors or shifts the whole scene at the start of a demonstration and replays the transformed end-effector path, training on both. Our counterfactuals are instead built at *arbitrary states of on-policy rollouts*, one change at a time (the target before the grasp, the container while carrying), and labeled by a privileged scripted teacher, in the spirit of learning from privileged experts [11] and on-policy distillation [1, 27]. We use them as an instrument to ask where VLAs fail, and treat the resulting training procedure as the minimal intervention the analysis calls for rather than as a data-generation contribution.

3. Does $\pi_{0.5}$ See the Displacement, and Does It Use It?

Model. $\pi_{0.5}$ [24] couples a PaliGemma VLM [7] (18 layers, width 2048) with an action expert (18 layers, width 1024) that produces a 50-step action chunk by flow matching [20]. At every layer the action tokens attend to the VLM’s prefix: 512 image tokens from an agent-view and a wrist camera, the instruction, and the proprioceptive state, which the LIBERO checkpoint writes into the prompt as about 120 digit tokens. The time step of the flow conditions the expert through adaptive RMS normalization. We use the public LIBERO checkpoint in its LeRobot port; on LIBERO-PRO it matches the numbers reported for the official model (swap: Object 18.5 vs. 18.2, Long 8.0 vs. 9.4 [10]).

Decision-time interventions. We take states s_t from successful rollouts of the policy and build a counterfactual state s'_t that differs in one respect, by editing the simulator’s kinematic state and re-rendering both cameras: (i) before the grasp, the target is moved by 3, 6, 12 or 20 cm to a free spot, or (ii) it swaps places with another object; (iii) while the object is carried, the container is moved by the same amounts; (iv) before the grasp, the instruction names another object of the scene; (v) only the proprioceptive input is offset by 5 cm. Nothing else changes, the robot included. States are binned by phase: *early* (hand farther than 15 cm from the target), *mid* (5–15 cm), *near* (closer than 5 cm) and *carrying*.

Use. Giving a state and its counterfactual the same flow-noise sample, we roll the first 25 steps of each predicted chunk through a kinematic model of the controller to obtain the hand’s intended displacement $m(s)$, and define

$$\text{use} = \frac{\langle m(s'_t) - m(s_t), \mathbf{d} \rangle}{\|\mathbf{d}\|^2}, \quad (1)$$

where $\mathbf{d}$ is the displacement of what moved (for (iv), from the target to the newly named object). A value of 1 means the motion follows the change fully, 0 that it ignores it. The supplementary material reports the metric for 10- and 50-step horizons and for several noise samples.

Encoding. For every layer we pool the hidden states of four streams (image tokens, instruction tokens, the last prefix token and the action tokens) and regress the *within-state* difference $h(s'_t) - h(s_t)$ on $\mathbf{d}$ with ridge regression, reporting cross-validated R^2 with folds grouped by episode. Differencing the same state removes everything that varies across states (where the hand is, the time in the episode), so the probe only asks whether the stream registers where the object was moved to.

Phase	3 cm	20 cm	swap	rename	proprio
early (>15 cm)	0.15	0.20	0.24	0.25	0.01
mid (5–15 cm)	0.59	0.34	0.04	0.02	0.01
near (<5 cm)	0.51	0.28	0.00	0.00	0.00
carrying	0.09	0.06	–	–	0.00

Table 1. **Use of a displacement by $\pi_{0.5}$** , LIBERO-Object (median over states). Columns: target moved by 3 or 20 cm (container while carrying), target swapped with another object, target renamed in the instruction, proprioception offset by 5 cm. 25-step horizon, one noise sample shared by a state and its counterfactuals. At the decision moments (bold) the action barely follows the change.

Findings. Table 1 reads as follows. (1) *At the decision moments the policy is nearly blind.* Early in the approach the action follows a moved target by about 20%; mid-way it ignores a swap (0.04) or a renamed target (0.02) entirely, and while carrying it follows a moved container by 6–9%: placing is a memorized motion. (2) *Once close, it is a local tracker.* Small displacements near the hand are followed (0.51–0.59) more than large ones, but the choice of object is locked. (3) *Proprioception is not the shortcut.* Offsetting the proprioceptive input has no effect in any phase, and replacing it by a constant at test time (Object swap 14.0, nominal 100) or during training (Spatial swap 40.5 vs. 40.0) does not change the failure. The same pattern holds on LIBERO-Spatial (early 0.21–0.30, swap 0.08–0.14, carrying 0.17–0.30) and LIBERO-Goal.

At the same time, the displacement is *encoded*. It is decodable from the image tokens at every layer (R^2 0.7–0.8 for a target moved by 12 cm or more, about 0.9 for a moved container) and is carried into the action tokens over the expert’s layers, from 0.05–0.2 in its first layers to 0.58 (target) and 0.74–0.82 (container) in its last ones. The attention of the action tokens is also nearly identical between blind and tracking phases (instruction 0.6–0.8 at a few layers, wrist image 0.2–0.3, agent image 0.1–0.3 in early layers), so the failure is not that the action tokens do not look at the image. $\pi_{0.5}$ knows where the target and the container are; at the moments that matter, its action does not depend on it.

4. Why Fixed Layouts Leave Use Unidentified

The gap between encoding and use is not an accident of $\pi_{0.5}$. We show that it is what the training risk permits when layouts are fixed, and derive what a remedy must satisfy.

Setup. Write the scene state as $s = (h, x, z)$: the robot state h , the position x of the object that decides the next motion (the target before the grasp, the container while carrying), and the rest of the scene z . The policy sees an observation $o = R(s)$ and the instruction ℓ ; the correct action

is $A(h, x)$. We assume (A1) the scene is *encoded*: there are maps $\hat{h}, \hat{x}$ with $\hat{h}(R(s)) = h$ and $\hat{x}(R(s)) = x$, as our probes indicate; and (A2) the action depends on x regularly: $m\|x - x'\| \leq \|A(h, x) - A(h, x')\| \leq L\|x - x'\|$ for some $0 < m \leq L$. A training distribution $\mathcal{D}$ has a *fixed layout with spread* σ if $x = g(\ell) + \eta$ with $\mathbb{E}\|\eta\|^2 = \sigma^2$: the instruction determines where the object is, up to η . In LIBERO, the target moves by 0–0.6 cm across the initial states of a LIBERO-Object task and by about 1 cm in LIBERO-Spatial and LIBERO-Goal. Consider the policies

$$\pi_\lambda(o, \ell) = A(\hat{h}(o), g(\ell) + \lambda(\hat{x}(o) - g(\ell))), \quad \lambda \in [0, 1], \quad (2)$$

which read the scene equally well but act on the observed position with weight λ ; λ is their *use* (Sec. 3): π_0 replays the memorized target, π_1 follows the scene.

Proposition 1 (Fixed layouts do not identify use). *Under (A1–A2), for a fixed layout with spread σ and every $\lambda \in [0, 1]$, $\mathcal{R}_{\mathcal{D}}(\pi_\lambda) = \mathbb{E}_{\mathcal{D}}\|\pi_\lambda(o, \ell) - A(h, x)\|^2 \leq (1 - \lambda)^2 L^2 \sigma^2$.*

All π_λ thus fit the data to within $L^2 \sigma^2$, and exactly when $\sigma = 0$: the training risk does not care whether the policy uses the position it encodes. Which λ is learned is then decided by inductive bias and initialization, not by the data. Two consequences follow directly.

Corollary 1 (Information is not identification). *Adding to the observation any channel $c = c(h, x)$, such as the exact displacement $x - h$ to the goal, leaves Proposition 1 unchanged: π_0 , which ignores c , still has risk at most $L^2 \sigma^2$.*

Now let $\mathcal{D}_{\text{cf}}$ be built from $\mathcal{D}$ by moving the decision variable, $x \mapsto x + \Delta$ with $\Delta \sim P_\Delta$, $\mathbb{E}\Delta = 0$, independent of (ℓ, h, z, η) , keeping h and z , and labeling with $A(h, x + \Delta)$. Train on the mixture $\mathcal{D}_\rho = (1 - \rho)\mathcal{D} + \rho\mathcal{D}_{\text{cf}}$.

Proposition 2 (Decision-time counterfactuals identify use). *Under (A1–A2), $\mathcal{R}_{\mathcal{D}_\rho}(\pi_1) = 0$ and, for every $\lambda \in [0, 1]$,*

$$\mathcal{R}_{\mathcal{D}_\rho}(\pi_\lambda) \geq \rho m^2 (1 - \lambda)^2 (\sigma^2 + \mathbb{E}\|\Delta\|^2). \quad (3)$$

Use becomes identified, with a margin that grows with the counterfactual mass $\rho \mathbb{E}\|\Delta\|^2$ rather than with the tiny layout spread. Proofs are in the supplementary material. The construction of $\mathcal{D}_{\text{cf}}$ matters, and each of its three conditions fails in a way that existing remedies illustrate.

(i) *The label must depend on the counterfactual.* If $\mathcal{D}_{\text{cf}}$ is labeled by a policy that itself ignores x , such as the base policy π_0 , then π_0 has zero risk on it: self-labeled counterfactuals reinforce non-use.

(ii) *Only the decision variable may change.* If the counterfactual transforms x and z together by a map T (mirroring the whole scene) and labels accordingly, a policy that reads T off z (the mirrored furniture, say) and applies T to

its memorized action fits $\mathcal{D}_{cf}$ without using x ; it identifies the response to T , not to x , and behaves like π_0 when only x moves.

(iii) *Identification is local.* The margin constrains the policy only on the states and displacements that $\mathcal{D}_{cf}$ covers: use before the grasp and use while carrying are identified separately.

Predictions. The analysis makes five predictions, which we test in Sec. 6: **(P1)** an oracle geometric input is ignored without counterfactuals (Corollary 1); **(P2)** self-labeled counterfactuals do not help (i); **(P3)** whole-scene transforms do not raise the use of a single object’s position, although the policy can learn the transformed scenes themselves (ii); **(P4)** counterfactuals before the grasp fix the choice of object and counterfactuals while carrying fix the placement, separately (iii); **(P5)** use grows with the counterfactual mass $\rho \mathbb{E} \|\Delta\|^2$ (Proposition 2). The analysis does not predict the value of an unidentified use: the partial following of small displacements near the hand that $\pi_{0.5}$ shows (Tab. 1) is such an inductive bias, a local visual alignment learned from the wrist camera, which does not extend to the choice of object.

5. Decision-Time Counterfactual Supervision

Proposition 2 says what a remedy must provide: training samples in which the decision variable moves while everything else stays, at the states where the policy decides, labeled with the action that is correct after the move. Decision-time counterfactual supervision (DCS) builds exactly such a $\mathcal{D}_{cf}$ inside the policy’s own rollouts; each of its components answers one condition of Sec. 4.

States: the policy’s own decisions. The student is $\pi_{0.5}$ with LoRA adapters [16] (rank 32) on the linear layers of the VLM and the action expert; the teacher is the same checkpoint with the adapters disabled. Each iteration, the student drives fresh rollouts, and at every queried state s_t the teacher labels the 50-step chunk A_t , as in DAGger [27], so that counterfactuals are placed on the states the policy actually visits (condition (iii)). The student is trained with the flow-matching loss of $\pi_{0.5}$, with $\epsilon \sim \mathcal{N}(0, I)$ and the flow time $\tau \in (0, 1]$ drawn from the Beta-shaped distribution of $\pi_{0.5}$ ($\tau = 1$ is pure noise),

$$\mathcal{L}_{\text{fact}} = \|v_\theta(o_t, x_\tau, \tau) - (\epsilon - A_t)\|^2, \quad x_\tau = \tau\epsilon + (1 - \tau)A_t. \quad (4)$$

Without counterfactuals this is our matched control (“Distill”); it leaves $\pi_{0.5}$ ’s behavior essentially unchanged, as Proposition 1 predicts.

Interventions: one decision variable at a time. At a state s_t , the simulator renders a world s'_t in which only the

variable that decides the next motion has moved (condition (ii)):

- *Retarget* (before the grasp): the target is moved to a free spot 8–30 cm away, or swaps places with another object; the hand and the rest of the scene stay.
- *Relocate* (while carrying): the container, or the furniture region the object goes to, is moved by 15–40 cm; hand and object stay.
- *Co-shift* (before the grasp), a control: hand and target move together, which leaves the decision unchanged and keeps the factual label correct up to the grasp.

Labels: privileged, and correct in the counterfactual world. The label of s'_t must be $A(h, x + \Delta)$, not the base policy’s output (condition (i)). We use privileged scripted teachers that know the moved position: an approach to above the target, valid for the steps it takes to get there (the grasp is left to the policy), and a placer to the new container, valid until shortly after the release. A scripted teacher is only as good as its assumptions, and a wrong label identifies a wrong use; three task-agnostic checks discard the pairs where the scripted notion of the decision variable does not hold:

- *Uniqueness.* An object or fixture with a twin of the same class is never moved: the instruction can then only refer to it by where it is (“the bowl next to the plate”), and moving it would change which object is meant.
- *Agreement.* In the factual world, the scripted teacher must head where the $\pi_{0.5}$ teacher heads (cosine at least 0.5 between their intended displacements over 25 steps), which rejects states where the task starts with another step or where the scripted target or container is wrong.
- *Phase.* Pre-grasp pairs only before the episode’s first grasp; relocate pairs only while an object of the task is held (gripper closed, object lifted and between the fingers).

Objective. The counterfactual observation o'_t and label A'_t , with a validity mask w over chunk steps, enter the same loss with the same noise and time,

$$\mathcal{L} = \mathcal{L}_{\text{fact}} + \frac{\sum w \odot \|v_\theta(o'_t, x'_\tau, \tau) - (\epsilon - A'_t)\|^2}{\sum w}, \quad (5)$$

on a share $\rho = 15\%$ of the states of an iteration; Proposition 2 makes $\rho \mathbb{E} \|\Delta\|^2$ the quantity that sets the identification margin. A pairwise consistency term on $v_\theta(o') - v_\theta(o)$ brought no gain and is not used, in line with [10]. Objects are recolored at random during the rollouts (probability 0.5 per episode), against a reliance on color that appears once the policy starts to use the image (Sec. 6.4).

Training budget. 20 iterations of 1024 states each, batch 8, learning rate $5 \cdot 10^{-5}$ with cosine decay, one suite at a

time; every compared method uses the same teacher, roll-outs, budget and optimizer. A run takes 5–7 hours on one H200 GPU, about half of it simulation and rendering; the counterfactual views add one render per queried state. Details and the scripted teachers are given in the supplementary material.

6. Experiments

Benchmarks. We train on one LIBERO suite at a time [21] and evaluate on its nominal tasks and on the LIBERO-PRO perturbations of the same suite [37]: *swap* (objects exchange places), *position* (the target is moved; LIBERO-Object only), *object* (objects recolored or re-textured) and *language* (instruction rephrased). Each cell is 10 tasks $\times$ 20 initial states; horizons are 280 (Object), 220 (Spatial), 300 (Goal) and 520 (Long) steps. Seeds are 7, 8, 9 unless stated.

Baselines, all in the same harness, teacher and budget. *Distill*: the on-policy distillation of Sec. 5 without counterfactuals. *ECT* [10]: each iteration, 16 successful episodes are replayed in a transformed scene (robot and start pose unchanged; Object: mirror in y , mirror in x and in xy with a shift; Spatial and Goal: mirror in y ; Long: a shift; furniture is reflected so that it faces the work area); a tracking controller follows the transformed end-effector path, and successful replays are paired with the original queries, on at most 15% of the states as for DCS. Pairs on 50% of the states do not help ECT (Object swap 25.0, Spatial 48.0). *3D-POINT* [30]: the 3D displacement from the gripper to the current sub-goal (the target until it is lifted, then its goal region), taken from the simulator at training and test time, is mapped by an MLP with a zero-initialized last layer and added to the flow-time conditioning of every action-expert layer; the teacher does not see it. Published numbers of these methods come from different training regimes (four suites jointly, 30k steps, or 69 tasks) and are not comparable to a per-suite adaptation; we therefore compare re-implementations.

6.1. Main results

Table 2 shows the outcome on the two suites where the swap failure is largest. DCS raises Object swap from 18.7 to 54.5 and Object position from 12.2 to 38.2, and Spatial swap from 40.0 to 65.0, keeping nominal success at 97–99. Under the same teacher and budget, ECT gains 4.5 and 10.2 points on the swap cells, DCS 35.8 and 25.0; the difference to ECT is 31.3 points on Object swap, 21.2 on Object position and 14.8 on Spatial swap (three seeds each). 3D-POINT does not move any cell. The cost is on the *object* cell of LIBERO-Object, where recolored or re-textured objects drop from 92.5 to 83.2; we analyze this trade-off in Sec. 6.4.

Which counterfactual matters (P2–P4). Table 3 varies the counterfactual and keeps everything else fixed. Transforming the whole scene, by mirroring with the equivariantly mapped action or by rotating the world about the robot, changes little, consistent with the per-suite ablations of ECT itself [10]. A direct test of (P3) evaluates the policies in mirrored LIBERO-Object scenes of the kind ECT trains on (seeds 7 and 8): $\pi_{0.5}$ succeeds in 11.0% of the episodes, ECT in 60.3%. ECT has learned the transformed scenes, which also confirms that our re-implementation works, yet its swap success stays at 23.2: it has learned to respond to the transform, not to the position of the target. DCS, which never sees a mirrored scene, succeeds in 54.3% of them: learning to use the position of one object transfers to transformed scenes, while learning the transforms does not transfer to a single object’s move. Co-shifting hand and target, which preserves the geometry the memorized action relies on, does not help either. What helps is a counterfactual that changes the decision: moving the target before the grasp fixes the choice of object (swap 53.0) but not the placement, and moving the container while carrying fixes the placement (position 32.8) but not the choice; combining them fixes both. The label matters as much as the world: on Spatial, labeling the relocated world with $\pi_{0.5}$ ’s own chunk instead of the privileged placer gives swap 40.0 (= Distill), against 64.7 with the placer (3 seeds), because $\pi_{0.5}$, as Sec. 3 shows, does not act on the relocation it sees.

6.2. What changes inside the policy

Table 4 applies the probes of Sec. 3 to the trained policies. The encoding is nearly the same in the three models. What differs is use: ECT leaves it at the level of $\pi_{0.5}$, while DCS makes the action follow a swap (0.03 $\rightarrow$ 0.53), a renamed target (0.02 $\rightarrow$ 0.54) and a moved container (0.06 $\rightarrow$ 0.44–0.69). Counterfactual supervision thus does not teach the policy to see; it teaches it to act on what it sees. One blind spot is shared by all three models: early in the approach, the action follows a moved target by only 0.13–0.30.

Explicit geometry is ignored (P1). 3D-POINT is the most direct test of whether information is missing: the policy receives the exact displacement to the sub-goal, including after a swap. Its use is identical to that of $\pi_{0.5}$ (Tab. 4). The injected branch converges to a near-constant offset: its output has a norm of about 1.6 but changes by only 0.05–0.08 when the displacement changes by 10 cm. On fixed layouts the goal point always coincides with the memorized trajectory, so nothing forces the policy to read it. Adding the counterfactual data of DCS to 3D-POINT (with a $10\times$ smaller learning rate for the branch, which otherwise destabilizes training) gives Object swap 43.0 and position 41.5, no better than DCS alone, and the branch is still ignored ($\|f(\mathbf{d}) - f(\mathbf{0})\|$ 0.03–0.12 for 10–30 cm against $\|f(\mathbf{0})\|$

Method	seeds	LIBERO-Object					LIBERO-Spatial			
		nominal	swap	position	object	language	nominal	swap	object	language
Distill (no counterfactual)	3 / 2	99.3	18.7 $\pm$ 2.8	12.2 $\pm$ 1.3	92.5	99.2	97.5	40.0 $\pm$ 2.8	98.5	96.5
3D-POINT [30] (oracle point)	1	98.5	18.5	12.5	93.0	100.0	99.0	40.0	98.0	97.0
ECT [10]	3	99.0	23.2 $\pm$ 1.0	17.0 $\pm$ 0.9	93.3	98.7	98.5	50.2 $\pm$ 1.6	97.5	96.3
DCS (ours)	3	97.3	54.5 $\pm$ 3.8	38.2 $\pm$ 5.6	83.2	97.3	98.8	65.0 $\pm$ 1.8	98.3	96.5

Table 2. **LIBERO-PRO success (%) after per-suite adaptation of $\pi_{0.5}$** , mean $\pm$ std over seeds (200 episodes per cell and seed). All methods share the teacher, rollouts, budget and optimizer. The untrained $\pi_{0.5}$ scores 99 / 18.5 on Object nominal / swap. Distill has 3 seeds on Object and 2 on Spatial.

Counterfactual (LIBERO-Object)	seeds	swap	position	nominal
none (Distill)	3	18.7	12.2	99.3
<i>whole scene transformed</i>				
mirror (equivariant label)	2	20.8	12.8	98.0
rotation about the robot	3	29.2	16.2	99.2
<i>hand-target geometry preserved</i>				
hand and target co-shifted	3	24.3	20.7	98.7
<i>decision changed, privileged label</i>				
retarget (before grasp)	3	53.0	20.5	99.0
relocate (while carrying)	2	22.5	32.8	98.8
retarget + relocate	3	56.3	23.7	96.8
retarget/co-shift + relocate	1	53.0	39.0	96.0

Table 3. **Which counterfactual helps.** Only counterfactuals that change what the policy must do at a decision state, labeled by the privileged teacher, raise the cells: retargeting fixes *which* object (swap), relocating fixes *where* it goes (position). Without validity checks or recoloring.

Probe (LIBERO-Object)	$\pi_{0.5}$	3D-POINT	ECT	DCS
early, target moved	.13-.22	.12-.21	.14-.23	.20-.30
mid, moved 20 cm	.31	.32	.37	.67
mid, swap	.03	.03	.07	.53
mid, target renamed	.02	.02	.01	.54
carrying, cont. ≥ 12 cm	.06	.06	.11	.44-.69
use				
target moved ≥ 12 cm	.58	—	.62	.65
swap	.55	—	.56	.70
container moved ≥ 12 cm	.80	—	.86	.85
enc.				

Table 4. **Use and encoding after training** (seed 7; DCS: the configuration of the last row of Tab. 3). Use: median follow of the action (Sec. 3); ranges span displacement sizes. Encoding: in-state probe R^2 of the action tokens, late expert layers. Every model encodes the change; only DCS acts on it.

Trainable part	params	swap	pos.	nominal	object
none (Distill)	—	18.7	12.2	99.3	92.5
expert readout	34K	23.5	20.5	100	92.5
expert, last 6 layers	4.7M	34.5	30.5	82.5	80.5
action expert (2 seeds)	13.9M	32.0	29.8	94.3	87.5
VLM keys and values	2.7M	20.0	9.5	72.0	63.5
VLM layers 9–17	19.6M	41.0	28.5	92.0	76.5
VLM (3 seeds)	39.2M	49.3	29.8	96.0	68.7
all		53.0	39.0	96.0	80.0

Table 5. **Localization.** LIBERO-Object, the counterfactual data of the last row of Tab. 3, only the listed LoRA adapters trained. Adapting the VLM with the action expert frozen gives 89% of the swap gain.

6.3. Where the change lives

With the counterfactual data fixed, we restrict which LoRA adapters may change (Tab. 5). The readout of the action expert, the natural place for a decoding bottleneck and the layer whose re-training suffices in image classification [19], gains almost nothing. Forcing the change into the last layers of the action expert gains part of the swap cell but breaks nominal behavior (82.5). The full action expert recovers about 40% of the swap gain and two thirds of the position gain. Adapting only the VLM, with the action expert frozen, recovers 89% of the swap gain at nominal 96, replicated over three seeds (50.0, 44.0, 54.0); most of it comes from layers 9–17 (41.0). Yet the change is not a re-presentation of what the action expert reads: adapting only the keys and values of the VLM, the exact interface the expert attends to, gives nothing and damages every cell. The fix requires the VLM to *process* the scene differently in its mid-to-late layers: deciding which object to go to is computed in the VLM, and $\pi_{0.5}$ learned to compute it from the instruction alone. Placement (position) needs both halves. The appearance trade-off also originates in the VLM: adapting only the VLM yields the lowest *object* cell (68.7), the action expert the highest (87.5).

1.08): under pressure to use the target’s position, the policy solves the task through its VLM rather than through the new channel.

Method (seed 7)	Goal		Long	
	nominal	swap	nominal	swap
$\pi_{0.5}$	96.5	28.0	93.5	8.0
Distill	97.0	27.5	96.0	8.0
ECT	92.0	26.0	83.0	12.5
DCS, as Tab. 2	89.5	30.5	83.0	19.5
DCS + scene-change phase	92.5	32.5	89.0	16.0

Table 6. **Multi-step suites**, with the furniture placement fixed per trial (Sec. 6.4). The last row adds the two phase rules found by the per-task analysis (pre-grasp pairs only once the rest of the scene stops changing; no swap with another object the task carries). **Same configuration on Object / Spatial: running.**

6.4. Costs, other suites, and an evaluation detail

Appearance. A policy that ignores the scene is robust to recoloring for free; a policy that looks at it can start to identify objects by color. In the VLM-only configuration, the validity checks of Sec. 5 raise the *object* cell from 69 to 83.5; recoloring objects during the rollouts adds only 1.5 points to it and costs 9 points of swap (48.5 $\rightarrow$ 39.5). **Main model without recoloring and ECT with it: running.** With the oracle point of 3D-POINT, the *object* cell returns to 92.5, as the point identifies the target regardless of color; claiming this would need a non-privileged source.

LIBERO-Goal and LIBERO-Long. On the multi-step suites (Tab. 6) the gains are smaller: DCS raises swap from 28.0 to 30.5–32.5 on Goal and from 8.0 to 16.0–19.5 on Long, more than ECT (26.0, 12.5), and both counterfactual methods lose nominal success. Per-task analysis traced the losses to label errors of the scripted teacher in multi-step tasks, each a violation of condition (i) of Sec. 4. First, $\pi_{0.5}$ opens the top drawer by pushing its handle without closing the gripper, so a gripper-based phase rule took the drawer phase for the approach to the bowl and labeled “go to the bowl!” there; starting pre-grasp pairs only once the rest of the scene stops changing removes these labels. Second, in “put both X and Y in the basket”, swapping the target with Y labeled “go elsewhere” while a valid object was under the hand; never swapping with another object the task carries removes them. With both rules, nominal success recovers to 92.5 (Goal) and 89.0 (Long), above ECT (92.0, 83.0), while staying below $\pi_{0.5}$. A remaining cause on Goal is the furniture. LIBERO *re-draws the furniture placement at every reset*: the cabinet moves by up to 2 cm in x and 1.4 cm in y , and the benchmark’s initial states, which store joint positions only, do not fix it. Pinning the cabinet at controlled offsets shows that DCS, unlike $\pi_{0.5}$ and Distill, is sensitive to it on the top-drawer task (Tab. 7): having learned to act on where movable objects are, it lost part of $\pi_{0.5}$ ’s ability to follow the furniture. **Counterfactuals that nudge the furni-**

Cabinet offset	$\pi_{0.5}$	Distill	ECT	DCS
none	90	95	90	80
−1 cm in x	95	95	80	65
−2 cm in x	85	90	85	60
(−2, −1.4) cm	95	90	85	55

Table 7. **Furniture sensitivity**, Goal task “open the top drawer and put the bowl inside” (20 trials, furniture pinned at the placement of a fresh environment plus the offset; offsets within the range LIBERO itself draws). DCS: the configuration with the best Goal nominal success in Tab. 6 (94.0).

ture and keep the teacher’s own response there, which $\pi_{0.5}$ handles reliably within this range, are running.

Evaluation protocol. The standard LIBERO loop runs the trials of a task sequentially in one seeded environment, so trial k sees the k -th furniture draw; parallel evaluation gives a trial a placement that depends on scheduling. One adapter scores 65–70% on “open the middle drawer” with sequential trials and 95% in fresh environments, failing on the same trials in repeated sequential runs. We therefore seed the environment per (task, trial) before each reset (used for Tabs. 6 and 7 and recommended); LIBERO-Object has no furniture and is unaffected.

7. Conclusion

$\pi_{0.5}$ fails on rearranged scenes not because it cannot see where objects are, but because it does not act on it at the moments that decide the task. Separating use from encoding with decision-time interventions makes this visible, explains why adding geometric information does not help, and points to the remedy: supervision in counterfactual worlds built at those moments, with labels that are valid there. The change it induces lives in the VLM’s mid-to-late layers rather than in the action pathway.

Limitations. Our labels come from privileged scripted teachers in simulation; they cover reaching and placing, not articulated or contact-rich steps, and their validity checks were refined by per-task failure analysis on the benchmark itself. Results are for one policy ($\pi_{0.5}$), one benchmark family, and per-suite adaptation with a small budget; on the multi-step suites the gains are small and nominal success stays below that of $\pi_{0.5}$, and using the scene costs robustness to appearance changes. The early-approach blind spot, shared by every model we measured, remains open, as does an intervention that targets the localized layers directly instead of adapting them with data.

References

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024. 3
- [2] Anonymous. Imagining recovery: Inference-time counterfactual realignment for vision-language-action models. *arXiv preprint arXiv:2608.14822*, 2026. TODO verify authors. 3
- [3] Anonymous. CounterAlign: Counterfactual supervision for vision-language-action models. *arXiv preprint arXiv:2608.21740*, 2026. TODO verify authors. 3
- [4] Anonymous. Encoded but not in control. *arXiv preprint arXiv:2610.06235*, 2026. TODO verify full title and authors. 2
- [5] Anonymous. Not all features are created equal. *arXiv preprint arXiv:2603.19233*, 2026. TODO verify full title and authors. 2
- [6] Anonymous. Flatness preserves instruction following in vision-language-action models. *arXiv preprint arXiv:2606.23641*, 2026. TODO verify authors. 3
- [7] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024. 3
- [8] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2
- [9] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. 1, 2
- [10] Chen et al. When instructions retrieve trajectories: Diagnosing and mitigating generalization failures in VLA models. *arXiv preprint arXiv:2609.39971*, 2026. TODO verify authors. 2, 3, 5, 6, 7
- [11] Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Conference on Robot Learning (CoRL)*, 2019. 3
- [12] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 2
- [13] Yu Fang, Yuchun Feng, Dong Jing, Jiaqi Liu, Yue Yang, Zhenyu Wei, Daniel Szafir, and Mingyu Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs. *arXiv preprint arXiv:2602.17659*, 2026. 3
- [14] Senyu Fei et al. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025. TODO verify authors. 2, 3
- [15] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. 2
- [16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 5
- [17] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. 1, 2
- [18] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 2
- [19] Polina Kirichenko, Pavel Izmailov, and Andrew Gordon Wilson. Last layer re-training is sufficient for robustness to spurious correlations. In *International Conference on Learning Representations (ICLR)*, 2023. 2, 7
- [20] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3
- [21] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023. 1, 2, 6
- [22] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning (CoRL)*, 2023. 3
- [23] Mohammad Pezeshki, Sékou-Oumar Kaba, Yoshua Bengio, Aaron Courville, Doina Precup, and Guillaume Lajoie. Gradient starvation: A learning proclivity in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 2
- [24] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 3
- [25] Silviu Pitis, Elliot Creager, and Animesh Garg. Counterfactual data augmentation using locally factored dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 3
- [26] Delin Qu et al. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. 2, 3
- [27] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. 3, 5
- [28] Harshay Shah, Kaustav Tamuly, Aditi Raghunathan, Prateek Jain, and Praneeth Netrapalli. The pitfalls of simplicity bias in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2
- [29] Haoyuan Shi, Xiancong Ren, et al. VLA-Trace: Diagnosing vision-language-action models through representation and

- 729 behavior tracing. *arXiv preprint arXiv:2605.30117*, 2026.
730 TODO complete authors. 2
- 731 [30] Tsai et al. Direct action-head injection of a grounded 3D
732 point. *arXiv preprint arXiv:2606.27663*, 2026. TODO verify
733 full title and authors. 2, 3, 6, 7
- 734 [31] Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman,
735 and Yang Gao. Fighting copycat agents in behavioral cloning
736 from observation histories. In *Advances in Neural Informa-*
737 *tion Processing Systems (NeurIPS)*, 2020. 2
- 738 [32] Siyu Xu, Zijian Wang, Yunke Wang, Chenghao Xia, Tao
739 Huang, and Chang Xu. Affordance field intervention: En-
740 abling VLAs to escape memory traps in robotic manipula-
741 tion. In *IEEE/CVF Conference on Computer Vision and Pat-*
742 *tern Recognition (CVPR)*, 2026. 3
- 743 [33] Zhengrong Xue, Shuying Deng, Zhenyang Chen, Yixuan
744 Wang, Zhecheng Yuan, and Huazhe Xu. DemoGen: Syn-
745 thetic demonstration generation for data-efficient visuomo-
746 tor policy learning. In *Robotics: Science and Systems (RSS)*,
747 2025. 3
- 748 [34] Zhihao Zhan, Qinhan Lyu, et al. Stable language guidance
749 for vision–language–action models. In *Annual Meeting of*
750 *the Association for Computational Linguistics (ACL)*, 2026.
751 TODO complete authors. 3
- 752 [35] Yan Zhang, Yinan Wu, Haoran Duan, and Jungong
753 Han. CofactVLA: Deconfounding vision-language-action
754 models via counterfactual intervention. *arXiv preprint*
755 *arXiv:2608.04396*, 2026. 3
- 756 [36] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang,
757 Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3D-
758 VLA: A 3D vision-language-action generative world model.
759 In *International Conference on Machine Learning (ICML)*,
760 2024. 2, 3
- 761 [37] Xueyang Zhou et al. LIBERO-PRO: Towards robust and fair
762 evaluation of vision-language-action models beyond mem-
763 orization. *arXiv preprint arXiv:2510.03827*, 2025. TODO
764 verify authors. 1, 2, 6

Knowing Where, Not Going There: Vision-Language-Action Policies Encode Displacements They Do Not Act On

Supplementary Material

8. Proofs

Proposition 1. For $(o, \ell) \sim \mathcal{D}$ with $o = R(h, x, z)$ and $x = g(\ell) + \eta$, (A1) gives $\pi_\lambda(o, \ell) = A(h, g(\ell) + \lambda\eta)$, while the label is $A(h, g(\ell) + \eta)$. By the upper bound in (A2), $\|\pi_\lambda(o, \ell) - A(h, x)\| \leq L\|(1 - \lambda)\eta\|$; squaring and taking the expectation gives $\mathcal{R}_\mathcal{D}(\pi_\lambda) \leq (1 - \lambda)^2 L^2 \sigma^2$. $\square$

Corollary 1. Let the observation be (o, c) with $c = c(h, x)$. The policies π_λ do not read c , so their risks are those of Proposition 1; in particular π_0 , which uses neither $\hat{x}(o)$ nor c , has risk at most $L^2 \sigma^2$. The same bound holds for $\pi'_\lambda = A(\hat{h}, g(\ell) + \lambda(\hat{x}_c - g(\ell)))$ that reads the position from c instead of o . $\square$

Proposition 2. On $\mathcal{D}$, $\pi_1(o, \ell) = A(h, x)$ by (A1), so its risk is zero; on $\mathcal{D}_{\text{cf}}$ the observation is $o' = R(h, x + \Delta, z)$, so $\pi_1(o', \ell) = A(h, x + \Delta)$, the label. For π_λ on $\mathcal{D}_{\text{cf}}$, $\pi_\lambda(o', \ell) = A(h, g + \lambda(\eta + \Delta))$ and the label is $A(h, g + \eta + \Delta)$; the lower bound in (A2) gives an error of at least $m(1 - \lambda)\|\eta + \Delta\|$. Since Δ is independent of η with $\mathbb{E}\Delta = 0$, $\mathbb{E}\|\eta + \Delta\|^2 = \sigma^2 + \mathbb{E}\|\Delta\|^2$, and $\mathcal{R}_{\mathcal{D}_\rho}(\pi_\lambda) \geq \rho \mathcal{R}_{\mathcal{D}_{\text{cf}}}(\pi_\lambda) \geq \rho m^2(1 - \lambda)^2(\sigma^2 + \mathbb{E}\|\Delta\|^2)$. $\square$

Remarks (i)–(iii). (i) If $\mathcal{D}_{\text{cf}}$ is labeled by π_0 , then $\mathcal{R}_{\mathcal{D}_{\text{cf}}}(\pi_0) = 0$ and $\mathcal{R}_{\mathcal{D}_\rho}(\pi_0) \leq (1 - \rho)L^2\sigma^2$: the memorizing policy remains a near-minimizer. (ii) Let the counterfactual apply a map T to the whole scene, $x' = Tx$, $z' = Tz$, with the robot fixed and label $A(h, Tx)$, and assume T is recognizable from z' (e.g. the mirrored furniture). The policy $\pi_T(o, \ell) = A(\hat{h}, \hat{T}(o)g(\ell))$, with $\hat{T}(o)$ the transform read off z (the identity on unmodified scenes), has error at most $L\|T\eta\|$ on $\mathcal{D}_{\text{cf}}$ and $L\|\eta\|$ on $\mathcal{D}$, so it fits the mixture to within $O(L^2\sigma^2)$ without reading x ; when only x moves, $\hat{T} = \text{id}$ and $\pi_T = \pi_0$. (iii) The bound of Proposition 2 involves only the states and displacements in the support of $\mathcal{D}_{\text{cf}}$; outside it, the argument of Proposition 1 applies unchanged.

Scope of the assumptions. (A1) idealizes what the probes measure (decodable, not perfectly decoded, positions). With a noisy decoder $\hat{x} = x + \nu$, independent zero-mean ν and a linear A , the risk of π_λ becomes $L^2[(1 - \lambda)^2 s^2 + \lambda^2 \mathbb{E}\|\nu\|^2]$ with $s^2 = \sigma^2$ on $\mathcal{D}$ and $s^2 = \sigma^2 + \rho \mathbb{E}\|\Delta\|^2$ on $\mathcal{D}_\rho$, minimized at $\lambda^* = s^2/(s^2 + \mathbb{E}\|\nu\|^2)$: the classical shrinkage of a noisy cue toward a prior. Under

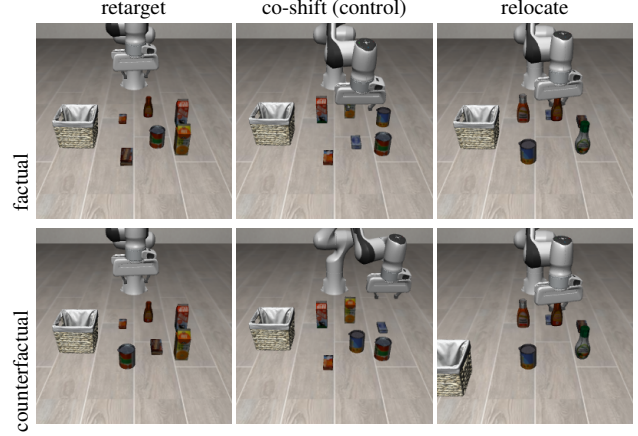


Figure 2. **Decision-time counterfactuals** at states of $\pi_{0.5}$'s own LIBERO-Object rollouts (agent view, as the policy sees it). *Retarget*: before the grasp, the target moves and the hand stays. *Co-shift*: hand and target move together, the decision is unchanged. *Relocate*: while carrying, the basket moves and hand and object stay.

a fixed layout ($s^2 = \sigma^2$ small) noise in the read-out position pushes the optimal use toward 0; counterfactual mass raises it toward 1. (A2) needs to hold only on the range of displacements considered, where the scripted approach and placement directions change monotonically with the target or container position.

9. Implementation details

Policy and training. LeRobot port of the $\pi_{0.5}$ LIBERO checkpoint; LoRA rank 32, $\alpha = 16$, no dropout, on all linear layers of the VLM language model and of the action expert (vision encoder frozen). Adam, learning rate $5 \cdot 10^{-5}$ with cosine decay to $5 \cdot 10^{-6}$, batch 8, gradient clipping. Each iteration collects rollouts of the student (20 episodes per batch, 8 parallel environments) until at least 1024 queried states are available, samples 1024 of them, and runs one epoch (128 steps). 20 iterations, i.e. 2560 optimizer steps. Every 4 iterations the adapter is evaluated on 10 trials per task (used only for monitoring). Rendering at 256×256 for both cameras.

Scripted teachers. The *approach* teacher moves the hand to a pre-grasp pose above the target (height from the target's bounding box) with the controller's maximal translation and the gripper open; its chunk is valid for the steps

it takes to get there. The *placer* lifts the held object to a safe height above the container’s rim, moves above the container’s opening (or the site of a furniture region), descends and releases; its chunk is valid until 5 steps after the release. Both act in the same action space as $\pi_{0.5}$ (end-effector deltas, gripper command) and are rolled out in a kinematic model to produce 50-step chunks.

Placing counterfactual objects. A moved target is placed at 8–30 cm from its position (or on the spot of another movable object, which takes the target’s spot, with probability 0.5), inside the area the objects occupy, without overlapping any other object’s bounding circle. A relocated container is moved by 15–40 cm, may leave the object area by up to 15 cm, and carries the objects lying in it. Furniture regions (top of a cabinet, a stove burner, a rack) are moved with their whole fixture, for rendering only.

Validity checks. Uniqueness uses the class name of every object and fixture. Agreement compares the sum of the first 25 translation commands of the scripted teacher in the factual world with that of the $\pi_{0.5}$ teacher (cosine ≥ 0.5 ; pairs where both barely move are kept). An *execution gate* that kept relocate pairs only on tasks whose placer succeeds in closed loop in at least 60% of 20 trials was used in earlier versions; it removed the tasks with the largest gains (e.g. Spatial swap 55.0 with the gate, 65.5 without) and is not used.

Compute. One run (one suite, 20 iterations) takes 5–7 hours on one H200 GPU, of which about half is simulation and rendering.

10. Robustness of the use metric and a second policy

The probes of the main paper compared a state and its counterfactual in different rows of a batch, which gave them different flow-noise samples. We re-ran them with one noise sample shared by every row, and report the use for 10-, 25- and 50-step horizons (Tab. 8). The values of the main paper are reproduced (25 steps), and the pattern holds at every horizon: swaps and renamed targets are ignored in mid-approach, moved containers are barely followed while carrying, small displacements near the hand are partly followed. Longer horizons give larger values throughout because the chunk then also contains the later part of the motion.

OpenVLA-OFT [18], a regression-head VLA with 8-step chunks, shows the same pattern at its horizon, more strongly: swap 0.01, renamed target 0.00 and moved container 0.00, with small displacements near the hand followed at 0.18 ($\pi_{0.5}$ at 10 steps: 0.02, 0.03, 0.02 and 0.36).

Policy, horizon	early		mid-approach			near	carrying
	3 cm	swap	3 cm	swap	rename	3 cm	3–20 cm
$\pi_{0.5}$, 10 steps	.00	.02	.32	.02	.03	.36	.02
$\pi_{0.5}$, 25 steps	.15	.24	.59	.04	.02	.51	.06–.09
$\pi_{0.5}$, 50 steps	.37	.33	.55	.07	.01	.44	.10–.19
OpenVLA-OFT, 8 steps	–.02	.01	.06	.01	.00	.18	.00

Table 8. **Use with a shared noise sample**, LIBERO-Object, median over states. **Second noise seed and trained models: running.**

11. Re-implemented baselines

ECT. Following [10], a transformed scene reflects the free objects and the furniture standing on the table (in x about the objects’ centre, in y about the robot base) and/or shifts them, while the robot and its start pose are unchanged. Furniture is reflected so that it keeps facing the work area. The end-effector path of a successful episode is mapped by the same transform and tracked by a controller that applies the transformed commands as feed-forward and corrects half of the tracking error at every step; rotation commands are mapped as pseudo-vectors and the gripper command is copied. Untransformed replays succeed in 90–100% of the episodes with this controller. Only successful replays are kept, and each is paired with the queries of its source episode. Transforms per suite follow the paper’s choices; the sizes of the shifts, not given there, are ours (–8 cm in x ; (–6, 6) cm with the double mirror).

3D-POINT. Following [30]: $\mathbf{d} = \mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{hand}}$ in the robot frame, where the goal is the target until it is lifted by 1 cm, then the region named by the task’s BDDL goal (on/in); an MLP $3 \rightarrow 1024 \rightarrow 1024$ with a zero-initialized last layer maps $\mathbf{d}$ to a vector added to the flow-time conditioning of every action-expert layer, at every step. $\mathbf{d}$ comes from the simulator at training and test time. The branch is trained with the LoRA learning rate; a $10\times$ larger rate made Object training collapse.

12. Additional results

Label failures found by per-task analysis. Per-task breakdowns of every cell located the remaining failures of the scripted labels, all of which we fixed with task-agnostic rules: (i) instructions that identify an object by its position (“the black bowl next to the plate”), fixed by the uniqueness check; (ii) tasks that begin with another step, e.g. opening a drawer before placing a bowl, fixed by the agreement check and by starting pre-grasp pairs only once the rest of the scene stops changing ($\pi_{0.5}$ pushes the drawer open without closing its gripper, so gripper events do not mark the step); (iii) tasks with two objects to carry, where swapping the target with the second object labels “go elsewhere” while a

917 valid object is under the hand, fixed by never swapping with
918 another object the task carries. Results of the configuration
919 with (ii) and (iii) on Goal and Long are running.

920 **LIBERO-Plus.** On the robot-initial-state and layout cat-
921 egories of LIBERO-Plus [14] (60 episodes each, LIBERO-
922 Object), DCS scores 88 / 77 and 83 / 80 (layout / robot,
923 seeds 7 and 8), ECT 83 / 78 and 87 / 77, Distill 85 / 83, 83
924 / 83 and 80 / 83: these categories are not affected.

925 13. Furniture placement in LIBERO

926 LIBERO builds each scene from a BDDL file whose place-
927 ment initializer samples the pose of every object and fixture
928 at reset. The benchmark’s initial states then overwrite the
929 joint positions, which hold the free objects and the robot,
930 but not the poses of fixtures attached to the world (cabi-
931 nets, stoves, racks), which live in the model. Consecutive
932 resets of one environment therefore place the furniture dif-
933 ferently: over 20 resets of LIBERO-Goal, the cabinet moves
934 within 1.9 cm in x and 1.4 cm in y , the stove and the rack
935 within 1.5–1.9 cm. A freshly built environment, seeded as
936 in the official evaluation, always uses the first draw. Simu-
937 lation state, controller state and the robot’s trajectory under
938 identical commands are bit-identical between a reused and
939 a fresh environment; only the images differ. Because the
940 official loop evaluates the trials of a task sequentially in one
941 environment, trial k is tied to the k -th draw; parallel evalua-
942 tion with workers that take over trials breaks this tie. Seed-
943 ing the environment’s random stream with $1000 \cdot \text{task} + \text{trial}$
944 before each reset makes the placement a function of the trial
945 alone; we verified that permuting the order of trials leaves
946 every placement unchanged. LIBERO-Object scenes have
947 no furniture.